



Game Development with Frameworks

Practical Skills Module 6: Topic 3

[Visit our website](#)

Practical Skills Module 6: Topic 3

Topic Code	PM-06-PS03
Topic	Use a known framework (of own preference) with functions to build a game

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, connected to framework/ environment of own choice, the learner must be able to:

- PA0301 Build a basic game using Python (compulsory)
 - Rock, Paper, Scissors Game
 - Number guessing game

Applied Knowledge

- AK0301 Concept, definition and functions of Python programming principles

Internal Assessment Criteria

- IAC0301 Expected results are achieved and the basic game is operable

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-06



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario: Python Game Development

You've been tasked with building a set of games to sharpen your coding skills and demonstrate your understanding of Python fundamentals, libraries, and game logic. Your goal is to implement two games, each in a separate Python file, organized in one folder. These games will vary in complexity and focus on different aspects of Python programming, including randomization and using a framework of your choice.

Create a folder named **python_games** and complete the following:

Step 1: Build a rock, paper, scissors game

- Create a Python file called **rps_game.py**.
- Build a basic Rock, Paper, Scissors game where the player chooses between three options, and the computer randomly selects one.
- Use Python's `random` module to generate the computer's choice.
- Write a function that compares the player's choice and the computer's choice to determine the result (win, lose, or tie).
- Display the result to the player.
- *Tips:*
 - You could create a graphical version of this game using the Kivy framework by importing `App` from `kivy.app`, and using `BoxLayout`, `Button`, and `Label` from `kivy.ui` to build the interface.

Step 2: Build a number guessing game

- Create a Python file called **guessing_game.py**.
- Use the `random` module to generate a number between 1 and 100.

- Decide on a fixed number of attempts for the player (e.g., 7 attempts).
- After each guess, display feedback:
 - “Too high!”
 - “Too low!”
 - “Correct!” and end the game if the guess is right.
- If the player runs out of attempts, display “Game Over” and reveal the number.
- Accept only valid integer guesses from the player. Handle invalid input gracefully (e.g., if the input is non-numeric, prompt the user again).
- *Tip:* For a graphical version, you could also use the Kivy framework by importing `App` from `kivy.app`, `BoxLayout` from `kivy.ui.layout`, `Button` from `kivy.ui.button`, `Label` from `kivy.ui.label`, and `TextInput` from `kivy.ui.textinput`.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
